

Protocol-Governed Systems: Closed-Loop Governed Evolution

(c) 2026 Bhash Ganti

Contact: <mailto:bachipeachy@gmail.com>

ORCID Profile: <https://orcid.org/0009-0007-3810-6520>

Status: v1 — revised for the all-structured pipeline and the authority-invariance result (PGS 0.6.1). The v0 edition is the published DOI record; this revision supersedes it as the working source.

Preface

This paper is part of the PGS technical paper series. The paper *Protocol-Governed Systems: Conceptual Model* established the architectural foundations: constitutional governance, the four-layer stack, and the separation of governance from execution. The paper *Protocol-Governed Systems: Compiler Conceptual Model* described how the compiler converts protocol declarations into a governed execution boundary called the Protocol Snapshot. The paper *Protocol-Governed Systems: Runtime Conceptual Model* described how the runtime consumes that snapshot and executes workflow instances without any domain knowledge. The paper *Protocol-Governed Systems: Architecture Inversion Concepts* established why inverting the traditional relationship between specification and implementation is a structural requirement, not a design preference. Together, those four papers establish that behavior is fully determined before execution begins and that the protocol is the sole source of behavioral truth.

This paper addresses the question those four left open: what happens when the protocol must change? Construction and execution are governed. Evolution has not yet been accounted for. This paper introduces the conceptual model for closing that loop.

Abstract

Protocol-Governed Systems govern construction and execution. The compiler determines what may exist. The runtime determines what happens when existence is realized. But systems must evolve. New requirements emerge. Domains expand. The protocol must change. If evolution is ungoverned, protocol sovereignty erodes at the moment it is most needed: at the boundary between what the system is and what it must become.

This paper defines the conceptual model for closed-loop governed evolution in PGS. It introduces the governed change pipeline — a staged progression from Change Request through Business Model, Business Intent, Governance Intent, and Design Intent to the Authoring Mandate — as the mechanism by which the protocol evolves without losing sovereignty. The Business Model serves as the canonical artifact of governed change; all downstream governance artifacts are projections of the Business Model rather than independently authored specifications. It establishes that nothing is ever greenfield: every change modifies a governed baseline, and if the Protocol Snapshot does not change, the system is invariant by definition. It explains why the traditional requirements phase leaks implementation decisions by design, and why the governed pipeline expressly prevents this through stage-enforced separation of concerns. It describes the canonical documentation set that makes governed evolution tractable — and that constitutes the complete, implementation-free oracle for any change agent, human or automated. It defines the Governance Dividend: the observable accumulation of architectural knowledge that governed evolution produces, and that makes each subsequent change cheaper to govern than the last. Finally, it establishes the pipeline’s defining structural property — **authority invariance with respect to the authoring actor**: because authority resides in governed artifacts, projection contracts, structural validation, approval gates, and compiler enforcement rather than in the actor producing the draft, the same loop may be authored by a human, a local model, or a frontier model without altering its governance. Authoring is interchangeable; authority is not.

1. Introduction

Every software system eventually faces the same moment: what was built must become something it is not yet. Requirements evolve. Domains expand. Gaps surface between the protocol that was designed and the behavior that is now needed.

Most systems handle this moment informally. A ticket is opened. A developer reads the existing code and infers what must change. Decisions are made locally, without ceremony, without a record of rationale. The system changes. The governance record does not. Over time, the gap between what was intended and what was built becomes archaeological: only through careful excavation of commit history, ticket threads, and informal documentation can a later engineer reconstruct why the system is the way it is.

PGS eliminates this problem at the construction and execution layers. The protocol is the sole source of behavioral truth. The compiler enforces admissibility before any execution begins. The runtime traverses only what the compiler constructed. There is no implementation-embedded business logic to diverge from declared intent.

But those properties hold only for what has already been governed. The moment a system must change, a new kind of problem appears: **who governs the act of change itself?**

If the answer is “no one in particular,” then the governed system is only as trustworthy as its last change process was careful. Protocol sovereignty is not an invariant of the architecture — it is a habit. Habits break under deadline pressure, organizational change, and accumulated technical debt.

The answer that this paper develops is different: **evolution is itself a governed protocol concern**. This is **closed-loop governed evolution**: the application of protocol governance to the process by which the protocol itself changes. The pipeline that produces protocol changes is subject to the same sovereignty principles that govern the protocol it produces. Change requests are governed artifacts. The pipeline that processes them enforces separation of concerns by stage. Governance decisions are captured as durable records, not as informal agreements. The result of the pipeline — the Authoring Mandate — is a governed input to the compiler, not an informal engineering decision.

This closes the loop. PGS governs construction. PGS governs execution. PGS governs evolution.

Protocol-Governed Systems are not just governed at rest. They are governed in motion.

2. The Open-Loop Problem

Traditional software development methodologies — waterfall, ITIL, Agile, and their variants — share a structural characteristic that is rarely named explicitly: **the SDLC is open-loop**.

Requirements are gathered. Specifications are written. Designs are produced. Implementation begins. The output of that implementation is a running system. But the running system is not systematically compared against the requirements that motivated it. Governance rationale decays. Design decisions become implicit. The specification and the system drift apart, and there is no governed mechanism to detect the divergence.

This is the open-loop problem: the output of the development cycle does not feed back into the governance of the next cycle in a structured, authority-bearing way.

The consequences are well understood in practice, though rarely traced to their root cause:

Requirements leakage: The requirements phase of any traditional methodology is not scope-bounded. There is no structural guard that prevents implementation decisions from appearing in requirements documents. A requirement that specifies “the system shall use a relational database” has already made a design decision. A requirement that specifies “the API shall return JSON” has already made a transport decision. These leakages are not failures of discipline — they are consequences of having no structural enforcement of separation of concerns at the requirements stage. Implementation decisions enter the specification layer because nothing forbids them.

Rationale decay: The reasoning behind a governance decision — why a boundary was drawn here and not there, why a constraint was imposed at this level and not another — is captured nowhere that the system can read. It lives in documents that age, in ticket threads that are archived, in the memory of engineers who eventually leave. The system carries the decision but not its justification.

Governance externalisation: In most methodologies, governance is a wrapper around engineering. A change control board approves tickets. An architecture review committee reviews proposals. But these governance acts are external to the system: they do not alter what the system knows about itself. The running system has no awareness of the governance that produced it.

Evolution amnesia: When the system must change, the baseline being changed is opaque. What was the system designed to do? What constraints were declared, and why? What decisions were deferred, and to which future change? These questions cannot be answered from the running system. They require archaeology.

PGS addresses the first three problems at the construction layer. The compiler enforces that protocol declarations are the sole source of behavioral truth. The runtime enforces that execution follows only what was compiled. The snapshot is the authoritative record of what the system may do.

But archaeology persists until evolution is governed. A PGS system with an ungoverned change process will accumulate the same rationale decay that traditional systems accumulate — only the implementation is governed; the decisions that produced it are not.

Closing the loop requires governing the evolution process itself.

3. SDLC Inversion — Established Ground

The PGS architecture inverts the traditional relationship between specification and implementation. This inversion is documented fully in the Architecture Inversion Concepts paper and is restated briefly here as the necessary context for what follows.

In traditional systems:

Implementation → Specification → System

The implementation makes behavioral decisions. The specification documents what was decided. The system is the implementation.

In PGS:

Protocol → Compiler → Snapshot → Runtime

The protocol declares behavioral intent. The compiler enforces admissibility and produces the executable boundary. The runtime executes that boundary without domain knowledge. The system is the protocol.

This inversion has a specific consequence for change: **there is no behavior in the system that was not declared in the protocol.** If you want to change the system's behavior, you change the protocol. The compiler validates the change. The runtime executes the new snapshot.

But the inversion also creates a gap. Traditional SDLC governs the process of writing implementation. PGS replaces that implementation with protocol. What governs the process of writing protocol?

The missing piece is an SDLC for protocol — one that is itself governed, that enforces separation of concerns by stage, and that produces a durable evidence chain from initial problem statement through to compiler-ready artifact specification.

That is what closed-loop governed evolution provides.

PGS inverted the architecture. Closed-loop governed evolution inverts the SDLC to match.

4. Nothing Is Greenfield

The first and most important principle of governed evolution is deceptively simple:

Nothing is ever greenfield. Every change modifies a governed baseline.

In traditional software, the concept of a “greenfield” project has real meaning: you start with an empty directory, make no assumptions, and build from scratch. Decisions are unconstrained. The design space is open.

In PGS, this situation does not exist — not even for the very first change request.

The PGS baseline is the Protocol Snapshot. The Protocol Snapshot defines, precisely and completely, what the system currently is: every admissible execution path, every declared capability, every governance boundary, every event, every storage policy. It is not a document approximating the system. It is the system.

Every Change Request modifies this baseline. Even the first Change Request — the one that adds the first workflow to an otherwise empty domain — is a change to the baseline. That baseline may contain only the substrate: governance boundaries, constitutional rules, federation declarations, execution concerns, and the compilation and attestation machinery. But it is a baseline, and the first CR changes it by adding to it.

The consequence of this principle is precise:

If the Protocol Snapshot does not change, the system is invariant by definition.

There is no such thing as a change to a PGS system that does not touch the Protocol Snapshot. A change that touches only documentation, only tooling, only infrastructure — but does not produce a new compiled snapshot — has not changed the system. It has changed things around the system.

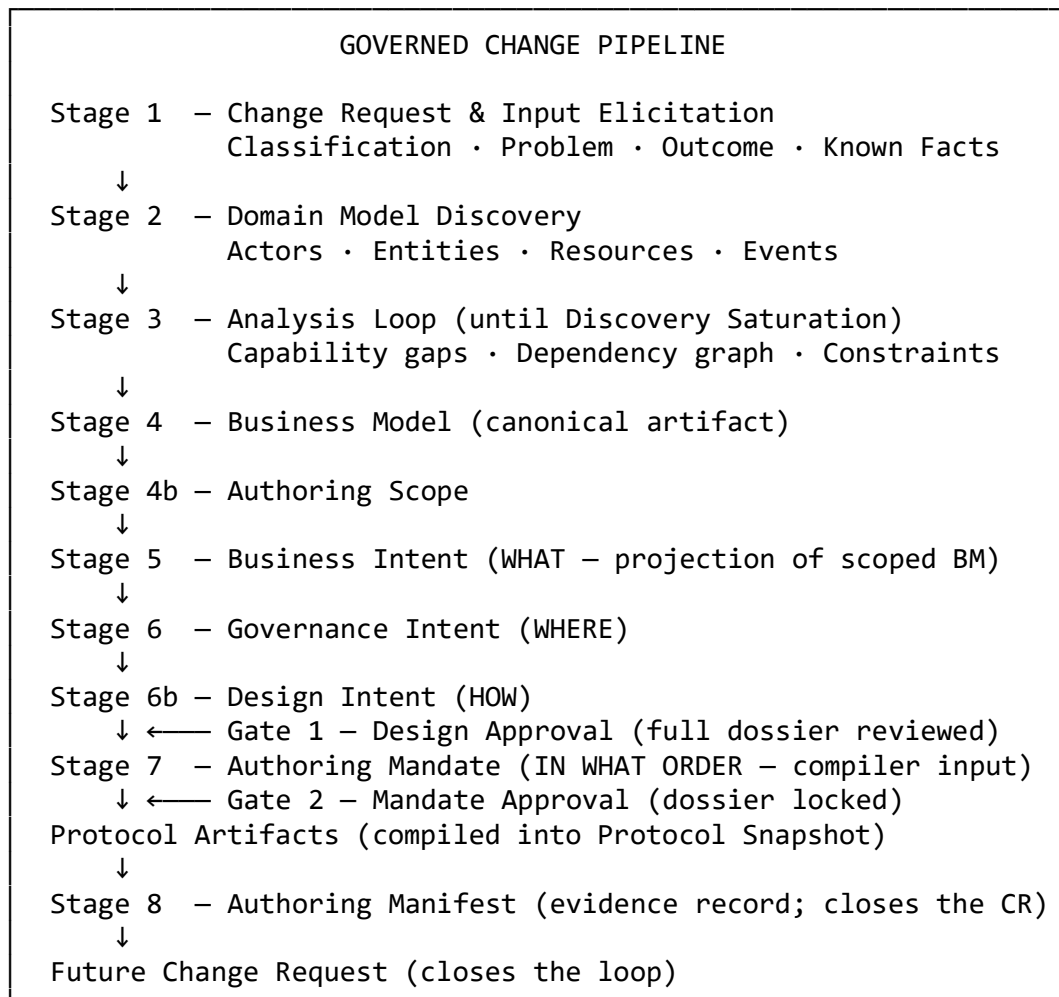
This precision matters for governed evolution. It establishes the stopping condition for any change: the process is complete when the Protocol Snapshot is updated, attested, and valid. Not when the documentation is written. Not when the implementation is merged. When the governed artifact changes.

System state:	Always defined by Protocol Snapshot
Change:	Any CR that produces a new Protocol Snapshot
No-change:	Any action that does not produce a new Protocol Snapshot
Invariance:	System is unchanged until Protocol Snapshot changes

This also clarifies the chain of authority: if you want to understand why the system is the way it is, you start from the Protocol Snapshot and work backwards through the change history. The dossiers — one per Change Request — are the evidence chain. They record not just what was decided, but the governed process by which the decision was reached.

5. The Governed Pipeline

The governed change pipeline is a staged progression from problem statement to compiler-ready artifact specification. Each stage is a cognitive gate — a bounded scope of inquiry that must be completed and reviewed before the next stage begins. Stages are not bureaucratic steps. They are separation-of-concerns boundaries enforced by the pipeline structure itself. In the current implementation each stage document is a **structured register document**: the authoring actor emits register rows, a deterministic renderer produces the document, and a **structural oracle** validates it mechanically — well-formed identifiers, controlled vocabularies, per-row traceability, and cross-stage consistency (for example, every code in the Authoring Mandate must trace to a Design-Intent register) — before any human review. Separation of concerns is thus enforced by the renderer and the oracle, not by convention.



Two structural readings of this diagram matter. First, classification and input elicitation are a single opening stage: classification has no content of its own beyond framing the elicitation. Second, the pipeline distinguishes **scope boundaries** (the

WHAT/WHERE/HOW question locks between Stages 5, 6, and 6b, enforced continuously as purity rules) from **approval gates** (the two human decision points at which the dossier is reviewed as a body and locked). Stages 1 through 6b form one iterative session; approval is sought when the design is whole, not per stage.

Stage Descriptions

Each stage has a precise cognitive scope — a defined question it answers, and a defined set of questions it is forbidden from answering. Understanding both sides of this boundary is essential for a practitioner executing the pipeline.

Two structural features run through every stage and are stated once here rather than repeated per stage:

The elicitation contract. Every stage opens with a small set of questions addressed to the human — crisp, answerable, and each paired with a declared intent: how the answer will be used in the document about to be drafted. This pairing is the prescriptive core of the pipeline. The practitioner is never asked to guess what the question is for, and the agent is never free to repurpose an answer beyond its declared intent. An unanswered question is an open gap in the dossier — it is never license to assume. The elicitation contract is what makes the division of labor explicit: the human supplies governed knowledge; the agent structures, verifies, and projects it.

Stage execution rules. Every stage carries verification rules that the executing agent must satisfy — claims about the baseline must be evidenced by reading the snapshot, not recalled from memory; new capabilities must be named in business language until the stage that assigns codes; nothing may be declared new before the existing inventory has been searched. These rules are not stylistic guidance. They are the accumulated failure knowledge of completed change cycles, folded back into the stage definitions so that every future change — and every future agent — inherits the lessons as enforced constraints rather than as folklore.

Stage 1 — Change Request & Input Elicitation Opens the dossier. One stage performs two inseparable acts: classifying the change and surfacing the problem in its raw form.

Classification determines the analysis path. Common classes include:

- Feature — adds a governed capability within one or more existing subdomains; no new subdomain declared
- Subdomain — extends or declares a subdomain for the first time; full governance pipeline required
- Domain — declares an entirely new governed domain; highest authority level
- Error/Bug — corrects a misauthored or non-conforming artifact in the existing snapshot
- None of the above — open classification; the pipeline still applies and the class is resolved within this stage

Elicitation then answers, in the human's own terms: the **Problem** (what is broken, missing, or ungoverned — stated in business language, never in artifact language), the **Outcome** (what governed capability must exist when the CR closes — the acceptance boundary against which closure is later judged), the **Known Facts** (what is already established at CR entry, with every claim about the existing baseline verified against the snapshot rather than trusted from memory), and the **explicit deferrals** (what is out of scope — because explicit deferral is a governance decision and silence is ambiguity). Contains no solution design, no topology decisions, and no new artifact proposals. The human drives this stage.

Stage 2 — Domain Model Discovery Identifies the structural elements the change touches: business entities (with the attributes that matter and the record character of each — current state, accumulated history, or stable identity binding), the business processes they participate in, and the current state of the governed baseline. The agent reads the snapshot directly — existing workflows, capability contracts, events, transforms, seed data, and store structures — and compares them against the CR's stated outcome. The baseline reading must record what was *searched*, not only what was found: capabilities the change needs frequently already exist under names the practitioner did not mention, and the most expensive discovery failures are re-authorings of things that already existed. Produces four outputs: a business entity model, a process description, a baseline fit assessment, and a gap analysis. Gaps discovered here become named questions in Stage 3. Contains no new capability design and no artifact codes for anything new.

This stage is structural and can be generated largely by an agent reading two inputs: the Stage 1 document and the current snapshot. Human review is still expected, though approval rigor here is lower than at the gates, and decreases further as the system and agent mature.

Stage 3 — Analysis Loop - For each open question from Stage 2, read the snapshot artifacts directly and compare what the CR needs against what already exists. Every answer carries its evidence — what was read and what it says. No assertions without evidence. - Every comparison produces one of two results: SATISFIED (capability exists and fits) or a gap. Every gap gets a resolution — reuse, update, or new — and every “new” must name the existing candidates that were examined and why each does not fit. - Design decisions are made here as gaps are resolved — choices like record shape, pattern selection, and option resolution belong in this stage, not later. - Iterate until no new gaps appear and no open questions remain. That is Discovery Saturation. - When a later iteration overturns an earlier answer — and verification passes against the baseline do overturn answers — the overturned answer is marked and retained, not erased. The loop's provenance is part of the evidence chain; the dossier records how the analysis converged, not just where it landed. - Output: gap register, constraint register, design decisions register. - New capabilities are named in business language. No artifact codes. No build order.

Stage 4 — Business Model - Consolidate everything Stages 1–3 produced into one coherent record. Consolidation, not re-litigation: nothing decided upstream is reopened

here. - Actors, entities, events, resources — the domain model. - Capability graph — what capabilities the CR needs and whether each is new, updated, or already satisfied. - Dependency graph — what depends on what, including ownership: a capability that must live in a peer subdomain is recorded as a gap owned by that peer, never as satisfied. - Constraint register — the non-negotiable rules, each with its business source. - Gap register — every gap with its resolution and its owning subdomain. - Design decisions register — every choice made in Stage 3, with rationale and the constraints it imposes downstream. - All downstream stages are read from this document. Nothing downstream may contradict it.

Stage 4b — Authoring Scope - A single boundary: which discovered capabilities are IN this CR, and which are deferred — with a stated reason for every deferral. - Every item that will be built or changed in this CR must appear here. Everything not listed is explicitly deferred. - This boundary is the CR's contract. If it is not here, it does not get built; deferred items become candidate inputs to future CRs.

Stage 5 — Business Intent - Translates the scoped Business Model into a structured behavioral declaration, authored in discovery order: purpose, scope boundary, business objects (what records exist and why they take the form they do), identity (which fields key them and what a duplicate means), invariants (what is always forbidden or required, and the business reason), business actions (what verbs can happen and what triggers each), actors (who is authorized), intents (what each action requires from the caller, and why each field is required), workflows (in what order the checks execute), and capability contracts (what each step does and what outcome it guards against). - Workflow is authored last, not first — it is an outcome of the earlier declarations, not a starting point. - Each in-scope business action maps to exactly one intent and one workflow; the derivation is mechanical because the business decision already happened. - Provisional capability names enter here — the structural vocabulary of intents, workflows, and contracts — but no binding identifiers, no file paths, no store paths, no module references, and no implementation bindings. Those belong to later stages. - The behavioral grammar is constrained by the execution model established in the prior papers: workflow steps route on declared outcomes; events record facts and never trigger execution; one workflow engages another only through a declared invocation step, never by embedding; and a record is written only by the subdomain that owns it. A Business Intent that violates the execution grammar describes a system that cannot be compiled — these constraints are checked here, not discovered at authoring time.

Stage 6 — Governance Intent - Declares WHERE the change lives. Which domain and subdomains this CR touches — and whether each is existing or new. Subdomain existence is a governance topology declaration, never derived from the snapshot. - Draw ownership boundaries: which capability belongs to which subdomain. The governing rule is that store ownership is a hard boundary — a capability that must write a peer subdomain's records is owned by that peer, declared as a dependency gap triggered by this CR. Cross-subdomain calls and reads are permitted and declared with explicit direction; cross-subdomain writes are forbidden without exception. - Declare storage governance requirements — which

subdomain owns which records — and the authority class under which operations execute. - List existing baseline artifacts that require action (update, replace, reuse). - No new artifact codes. No store paths. No build order.

Stage 6b — Design Intent - Declares HOW the change is expressed. Resolve every design decision from the Business Model into a concrete choice — every resolution traces back to a registered decision, and a decision invented here is flagged as such. - Assign the binding identifier and artifact family for every new capability declared in the Governance Outcome. - Declare the execution topology for every new or changed workflow, the pipeline steps of every contract, the schemas of every intent, and the stores with their shapes and ownership. - Verify field-level facts against the compiled baseline — what a producing step actually outputs, not what it is assumed to output. Every consumed field names its source. - Reconcile: the artifact summary must equal the declared artifact set exactly. A count that does not reconcile means something was silently added or dropped. - No build order. No implementation bindings — those are authoring-phase detail. - **Gate 1 — Design Approval** closes here: the full dossier (Stages 1–6b), including any upstream documents amended during the iterative session, is reviewed as one body and approved as the design basis.

Stage 7 — Authoring Mandate Declares IN WHAT ORDER. The governance-approved, compiler-ready specification: exactly what must be authored, in a build sequence derived by topological sort of the artifact dependency graph established in Design Intent. The build order is not an editorial preference — it is the dependency-determined order in which the compiler can validate each artifact against its declared predecessors. This stage is mechanical by design: it re-derives, it does not decide. The mandate must reconcile exactly with the Design Intent — same artifacts, same actions, same counts; if something looks wrong here, the Design Intent is fixed and the mandate re-derived. **Gate 2 — Mandate Approval** closes here: the dossier is locked, and any subsequent departure is a recorded deviation, never a silent change.

Stage 8 — Authoring Manifest The evidence record that closes the dossier. Created as an empty baseline when Gate 2 closes — before authoring begins — and populated during and after authoring and compilation. Records what was mandated, what was produced, approved deviations with their rationale, discoveries (architectural, implementation, vocabulary, and surface-alignment), conformance test results, and final snapshot state. Its closing section carries the methodology lessons of the cycle — the corrections that feed back into the stage definitions themselves. The manifest moves from draft to approved only when its completion criteria are met with actual execution data, never aspirationally. Manifest approval is CR closure.

The Dossier: Unit of Governed Change

Each Change Request produces a **dossier** — a single evidence chain containing every stage document for that CR. The dossier is not a document type hierarchy (requirements folder, design folder, implementation folder). It is one governed artifact whose documents are stages of a single inquiry.

dossiers/ <domain>/ <subdomain>/ change_request_<subdomain>_v0.md	← Stage 1	Classification + problem, outcome, known facts
domain_model_<subdomain>_v0.md	← Stage 2	Actors, entities, resources, events
analysis_loop_<subdomain>_v0.md	← Stage 3	Gap analysis, dependency graph, saturation
business_model_<subdomain>_v0.md	← Stage 4	Canonical artifact (all stages project from this)
business_intent_<subdomain>_v0.md	← Stage 5	WHAT: behavioral projection of BM
governance_intent_<subdomain>_v0.md	← Stage 6	WHERE: domain, subdomain, boundaries
design_intent_<subdomain>_v0.md	← Stage 6b	HOW: identifiers, topology, schemas – Gate 1
authoring_mandate_<subdomain>_v0.md	← Stage 7	IN WHAT ORDER: compiler input – Gate 2
authoring_manifest_<subdomain>_v0.md	← Stage 8	Evidence closure: results + snapshot state

The dossier is the answer to archaeology. When a future Change Request needs to understand why the system is the way it is, the dossier for the CR that produced it is the authoritative record — not commit messages, not ticket threads, not informal documents.

Scope Boundaries and Governance Decision Gates

The pipeline enforces two distinct kinds of boundary.

Scope boundaries are semantic. The question the pipeline is permitted to answer changes between stages, and the prior answer class is locked:

Boundary (between Stage 5 and Stage 6)

Closes: Business Intent – WHAT the system must do

Unlocks: WHERE questions – domain, subdomain, ownership become answerable

Locks out: binding identifiers, store paths, build order

Boundary (between Stage 6 and Stage 6b)

Closes: Governance Intent – WHERE the change lives

Unlocks: HOW questions – identifier assignment, topology, schemas, versioning

Locks out: build order, implementation specifics

Boundary (between Stage 6b and Stage 7)

Closes: Design Intent – HOW the change is expressed

Unlocks: IN WHAT ORDER – the build sequence is now derivable
Locks out: no new questions; the mandate is a derivation, not a decision

Scope boundaries are enforced continuously as purity rules — a Governance Intent that assigns artifact codes is a purity violation the moment it is written, not at a later review. They do not require a human decision; they require conformance.

Governance Decision Gates are human approval points, and there are exactly two:

Gate 1 – Design Approval (closes Stage 6b)

The full dossier (Stages 1-6b) is reviewed as one body – including any upstream document amended during the iterative session. Approval establishes the dossier as the design basis and authorizes Stage 7.

Gate 2 – Mandate Approval (closes Stage 7)

The Authoring Mandate is approved and the dossier is locked. Artifact authoring may begin. Any subsequent departure from the mandate is a recorded, approved deviation in the manifest – never a silent change.

Stages 1 through 6b are one iterative session: later stages routinely develop knowledge that amends earlier documents, and forcing a per-stage approval would either freeze stages prematurely or reduce approval to ceremony. The design is therefore reviewed when it is whole. Gates prevent premature commitment in the direction that matters — toward the compiler. A governance boundary drawn before the business model is complete is a guess; a design approved page-by-page is a design no one has seen whole. The two-gate structure keeps each stage honest about its scope while keeping human judgment positioned where it is decisive — and it makes the Authoring Mandate trustworthy when it reaches the compiler.

Discovery Saturation

The Analysis Loop (Stage 3) continues until **Discovery Saturation** — a precise stopping condition defined by three simultaneous properties:

1. No unresolved CRITICAL gaps remain
2. No open analyst questions remain
3. The dependency graph did not expand in the last review pass

Discovery Saturation is not declared by a person deciding the analysis is complete. It is a structural observation: when new analysis produces no new dependencies, no new gaps, and no new questions, the model is saturated. This matters because premature saturation produces incomplete Business Models, which propagate inconsistencies through every downstream stage.

6. Separation of Concerns — By Stage Design

The most important structural property of the governed pipeline is that it enforces **separation of concerns by stage**. Each stage has a precisely defined scope: what kind of question it answers, and what kinds of questions are forbidden from entering it.

Stages 1–4 – Discovery	answer: WHAT IS THE PROBLEM? WHAT EXISTS? permitted: business language; existing baseline artifacts cited as verified evidence forbidden: invented names for anything new; solution design; build order
Stage 5 – Business Intent	answers: WHAT? permitted: behavior, rules, constraints, actors, events; provisional capability names forbidden: binding identifiers, store paths, module references, implementation
Stage 6 – Governance Intent	answers: WHERE? permitted: domain, subdomain, ownership, federation boundaries forbidden: new artifact codes, store paths, build order
Stage 6b – Design Intent	answers: HOW? permitted: binding identifiers, artifact families, topology, schemas, versioning forbidden: build order, implementation bindings
Stage 7 – Authoring Mandate	answers: IN WHAT ORDER? permitted: topologically sorted build sequence forbidden: none – all upstream questions are closed

The separation forms a **purity ladder**: each rung admits one more vocabulary class than the rung above it. Business language only, through discovery. Provisional capability names at Business Intent — the structural vocabulary of the behavior, without binding. Placement without naming at Governance Intent. Binding identifiers at Design Intent. Order at the Mandate. At every rung, one exception holds uniformly: artifacts that *already exist* in the governed baseline may be cited by their exact identifiers as evidence, because citing the baseline is observation, not design.

This is not stylistic guidance. It is enforced by the pipeline structure. A Governance Intent document that assigns new artifact codes is a GI Purity violation. A Business Intent document that contains store paths or implementation bindings is a BI Purity violation. These are detected and corrected before the dossier advances.

The traditional requirements phase has no equivalent enforcement. A requirements document in any traditional methodology can contain architectural decisions, implementation preferences, technology choices, and deployment constraints alongside business rules — because there is no structural mechanism that forbids it. The consequences accumulate invisibly: when the implementation departs from the business rules, it is not always clear whether the requirement was wrong, the design was wrong, or the implementation was wrong. The levels were never separated.

In the governed pipeline, the separation is structural. By the time the Business Intent is closed and the Governance Decision Gate is passed, the document contains only business rules and behavior. There is nothing in it to confuse with implementation. When the compiler produces an artifact that diverges from stated intent, the divergence is unambiguous: the business intent was one thing; the artifact is another. Fix the artifact.

The final product of the pipeline — the Protocol Snapshot — contains **zero implementation details**. It contains only governed declarations: execution topology, capability contracts, routing conditions, admission rules, event identifiers, and storage access policies. Every one of those declarations was validated against a stage that declared only business behavior. The chain of authority is unbroken.

The pipeline has a second structural property that deserves explicit statement: it **methodically enriches** toward the final protocol artifacts in a manner that remains aligned with the baseline at every stage. Each stage adds a new layer of governed knowledge — behavioral (Stage 5), jurisdictional (Stage 6), structural (Stage 6b), sequential (Stage 7) — without ever retroactively contradicting what was established upstream. The enrichment is cumulative and coherent: the Business Model is the seed; the Authoring Mandate is the fully governed fruit.

This progressive enrichment is irreducibly human-driven. No machine can infer what business rules a domain must enforce, what governance boundaries are appropriate for a given organizational context, or what stakeholder intent actually underlies a Change Request. These are creative acts of domain understanding — not derivations from existing data. The elicitation contract makes this division explicit at every stage: the questions only the human can answer are stated up front, each paired with the declared intent for its answer, so the boundary between supplied knowledge and structured projection is visible in the dossier itself. A machine or LLM agent participating in the governed pipeline occupies a **supervisory and validation role**: eliciting against the contract, checking purity constraints, verifying every baseline claim against the snapshot, detecting saturation conditions, and confirming that the output aligns with the protocol baseline. It does not perform the creative governance reasoning that produces each stage's content. That belongs to the practitioner.

This distinction matters for how the pipeline is understood and deployed: the practitioner is not providing inputs to an automated system. The practitioner is the source of governed knowledge. The pipeline is the structure that captures, enforces, and preserves it.

The traditional requirements phase leaks implementation by accident. The governed pipeline excludes implementation by structure.

7. The Canonical Documentation Set

Governed evolution requires a precise answer to the question: **what constitutes the complete, authoritative view of the system being changed?**

In traditional systems, this question has no clean answer. The system's behavior is distributed across implementation code, database schemas, configuration files, infrastructure definitions, and informal documentation. To understand what the system currently does, you must read all of it — and read it with the interpretive knowledge to distinguish accidental implementation details from intentional behavioral decisions.

In PGS, the answer is exact. The canonical documentation set is:

- | | |
|----------------------|---|
| 1. Protocol Snapshot | – what the system currently does |
| 2. PPS Snapshot | – the full inventory of compiled protocol declarations |
| 3. Field Manual | – operational doctrine; how the system is run |
| 4. Concept Papers | – architectural rationale; why the system is built this way |

Nothing else is required. Specifically, no implementation source code is required. No database schema. No infrastructure configuration. No deployment manifest. These are substrate details, not protocol truth.

This set has a critical property: **it contains no implementation details.** The Protocol Snapshot declares admissible executions. It does not say what language they are implemented in, what hardware they run on, or what cloud provider hosts them. The field manual explains operational behavior. The concept papers explain architectural decisions.

A note on Protocol Snapshot vs. PPS Snapshot: These are distinct instruments serving different phases of the pipeline. The Protocol Snapshot is the runtime-executable artifact — the compiled, attested boundary that the runtime traverses. It is the authoritative answer to “what can the system currently do?” The PPS Snapshot is the governance-readable full declaration set: the complete inventory of artifact codes, FQDN identifiers, capability families, dependency relationships, and governance boundaries that currently exist in the protocol. It is the authoritative answer to “what currently exists that a new change must account for?”

During the behavior-definition phases (Stages 1 through 5), the practitioner primarily consults the Protocol Snapshot and prior dossiers to understand the baseline being changed. But from the Authoring Mandate forward — when the question shifts from “what should the system do?” to “what artifacts must be authored, and do they already exist?” — the PPS Snapshot becomes the primary reference. It is the inventory against which the Authoring Manifest is reconciled: every mandated artifact is checked against the PPS Snapshot to determine whether it is net-new, a replacement of an existing artifact, or a modification that requires a version increment. This makes the PPS Snapshot particularly valuable in the post-behavior, pre-compiler phase of the pipeline.

A change agent — a governance engineer, a domain expert, or an automated agent — working from this set has everything needed to reason about what the system currently does, why it was built that way, and what a proposed change must preserve or modify. The set is complete for the governance task, because governance does not operate on implementation. It operates on protocol.

This is why agent engagement is structurally well-suited to the governed pipeline. Not because AI is required — the pipeline works without AI — but because the canonical documentation set is the exact interface an agent needs. The context is bounded. The scope is declared. The separation of concerns is enforced by stage. A well-constructed agent, given this set and a Change Request, can execute every stage of the pipeline with authority.

Whether the agent is a human governance engineer, an LLM, or a future hybrid is irrelevant to the model. The pipeline governs the output of the stage, not the method by which the stage was executed.

The canonical set is the complete oracle for governed change. Its absence of implementation details is not a gap — it is the property that makes it an oracle.

8. Governance Produces Architectural Knowledge

A consequence of the governed pipeline that is not immediately obvious from its description: **the pipeline does not merely document decisions that were made outside it. It produces architectural knowledge that would not have been produced otherwise.**

This distinction matters. In traditional governance, a change control board reviews proposals that engineers have already designed. The governance act is a filter — it approves or rejects what was brought to it. The architectural decisions were made before the governance act. The governance act records outcomes, not reasoning.

In the governed pipeline, the stages are the reasoning process. Governance Intent does not record that a domain boundary was drawn; it draws the boundary, through the structured inquiry that the stage requires. Design Intent does not record that an artifact family was chosen; it determines the choice, through the mapping discipline that the stage enforces.

The consequence appears in practice: constraints that collide during the pipeline produce new architectural knowledge that was not present in the Change Request.

The first full PGS change cycle — the consensus_pos Change Request — produced the following architectural knowledge that was not specified in the CR:

- **GI Purity Rule:** Governance Intent must not contain artifact family codes. This was discovered when the initial GI draft included CC_ references. The pipeline detected the violation. The rule was formalized and applied to all future CRs.
- **Block ownership clarification:** The blockchain::block subdomain is a cross-consensus entity. Both PoS and future PoW depend on block. Governing block under consensus_pos would invert the dependency. This was discovered during the Governance Intent stage and resolved by declaring block as a peer subdomain.
- **Federation onboarding protocol:** The rules for admitting a new actor into the blockchain identity registry were not fully specified in the CR. The Analysis Loop surfaced the gap. The Business Model captured it.
- **Authoring Manifest concept:** The concept of a post-compilation evidence record — separate from the Authoring Mandate — was not in the original pipeline design. It emerged as a necessary artifact to close the evidence chain between the compiler output and the dossier.

None of these were specified. All were produced by the governed process. This is Governance as Discovery: the pipeline is not a transcription mechanism for decisions made elsewhere. It is the reasoning environment in which architectural knowledge is generated.

Subsequent cycles have continued the pattern, and the knowledge they produce is increasingly doctrinal rather than topological — rules about how change itself must be reasoned about:

- **Events record facts; they never trigger execution.** A change cycle that designed an event-driven trigger discovered, on verification against the baseline, that the execution model has no subscription mechanism — workflows engage other workflows only through declared invocation steps. The rule was formalized into the stage definitions.
- **Store ownership is a hard boundary, resolved by the dependency-gap pattern.** When a change requires writing records owned by a peer subdomain, the writing capability is owned by that peer and declared as a dependency gap triggered by the CR. The change crosses the boundary by declared call, never by write.
- **Search before authoring.** A capability the change needs may already exist under a name the practitioner did not use — in one cycle, the lifecycle event a change set out to create already existed in the baseline, reserved for exactly that change. The inventory check became an enforced stage rule.

This is the feedback mechanism that makes discovery cumulative: each cycle's Authoring Manifest carries its methodology lessons forward, and those lessons are folded into the stage definitions themselves. The stage templates are therefore not static forms — they are the accumulation vehicle for everything governance has learned about governing. The conceptual model in this paper is stable; the templates that operationalize it are expected to grow sharper with every cycle, and that growth is itself governed.

Governance does not record architectural decisions. It produces them.

9. The Governance Dividend

Governed evolution accumulates a property over time called the **Governance Dividend**: each completed change cycle makes subsequent changes easier to govern.

The dividend has three components:

Predictability: A governed change produces a durable, structured record. When the next change must understand what the current state is and why, the dossier answers both questions precisely. The cost of baseline understanding decreases with each completed CR.

Reduced change surface: The Authoring Mandate's build order is topologically sorted by artifact dependency. Only artifacts that are downstream of the change need to be authored or updated. The compiler enforces this: it recompiles only what changed. The scope of each change is bounded by the protocol graph, not by the engineer's knowledge of the codebase.

Architectural leverage: Each CR that enforces GI Purity, BI Purity, and Discovery Saturation produces a cleaner baseline for the next CR. The governance rules that emerged from the first cycle apply to all subsequent cycles. The model matures. The pipeline becomes more efficient as it accumulates validated patterns.

Empirical Basis: Completed Change Requests

As of this writing the governed pipeline has carried seven Change Requests in the blockchain domain — six executed through authoring into the compiled snapshot (consensus_pos, block, data_model, consensus_propose, mempool, orchestration) and the seventh (chain) at design approval. The first three established the pipeline's foundational rules and are described here; the later cycles are the source of the doctrinal discoveries reported in Section 8.

blockchain::consensus_pos — governed the Proof-of-Stake consensus mechanism: validator registration, staking and unstaking, reward and slashing policies, pool management, and the full block formation path from mempool transaction to proposed block. This was the first complete governance cycle and the one from which the pipeline's foundational rules — GI Purity, BI Purity, Authoring Manifest — were discovered and formalized.

blockchain::block — governed the block structure as a cross-consensus entity: block formation, attestation, finalization, and the data model for a block independent of the consensus algorithm that produces it. Declared as a peer subdomain rather than nested under consensus_pos because both PoS and future consensus algorithms depend on block. The subdomain boundary itself was a governance discovery from the consensus_pos cycle.

blockchain::data_model — governed the blockchain-wide data model: canonical entity definitions, field schemas, and relationship structures shared across all blockchain subdomains. This CR established the structural baseline that all future blockchain CRs must align with.

These CRs are not isolated features. They are interdependent subdomain governances that collectively define how the blockchain domain manages state, transitions, and events. Their successful completion through the full pipeline provides the empirical basis for the Governance Dividend claims that follow.

Case Study: consensus_pos Change Request

The first complete PGS governance cycle — the blockchain::consensus_pos Change Request — produced the following measured outcomes:

Authoring Mandate:	16 mandated authoring actions
Conformance tests:	77/77 PASS
New snapshot state:	VALID
Net new artifacts:	+3 protocol artifacts (above original scope)

The three additional artifacts — produced because the governance process discovered gaps not in the CR — are the observable Governance Dividend from the first cycle. They were not additional scope added by engineering judgment. They were architectural requirements that the governed process surfaced and captured.

Since the consensus_pos CR, five additional Change Requests have been executed through the pipeline, and a seventh is at design approval. Across these cycles the Governance Dividend is accumulating in observable forms: the purity rules are internalized, the canonical documentation set is richer, and the stage definitions themselves have absorbed each cycle's methodology lessons — so that a change agent entering the pipeline today inherits, as enforced structure, everything earlier cycles learned the hard way.

The comparison to traditional SDLC KPIs is instructive. Traditional methodologies measure governance cost as overhead: the cost of review meetings, approval cycles, and documentation. The Governance Dividend inverts this framing: **governance is not a cost center for change. It is an investment in the quality of the next change.**

The observed Governance Dividend is empirical but currently based on a limited number of completed governance cycles; future cycles will further validate or refine the model.

Architectural Objective: Coverage Lowers Change Cost

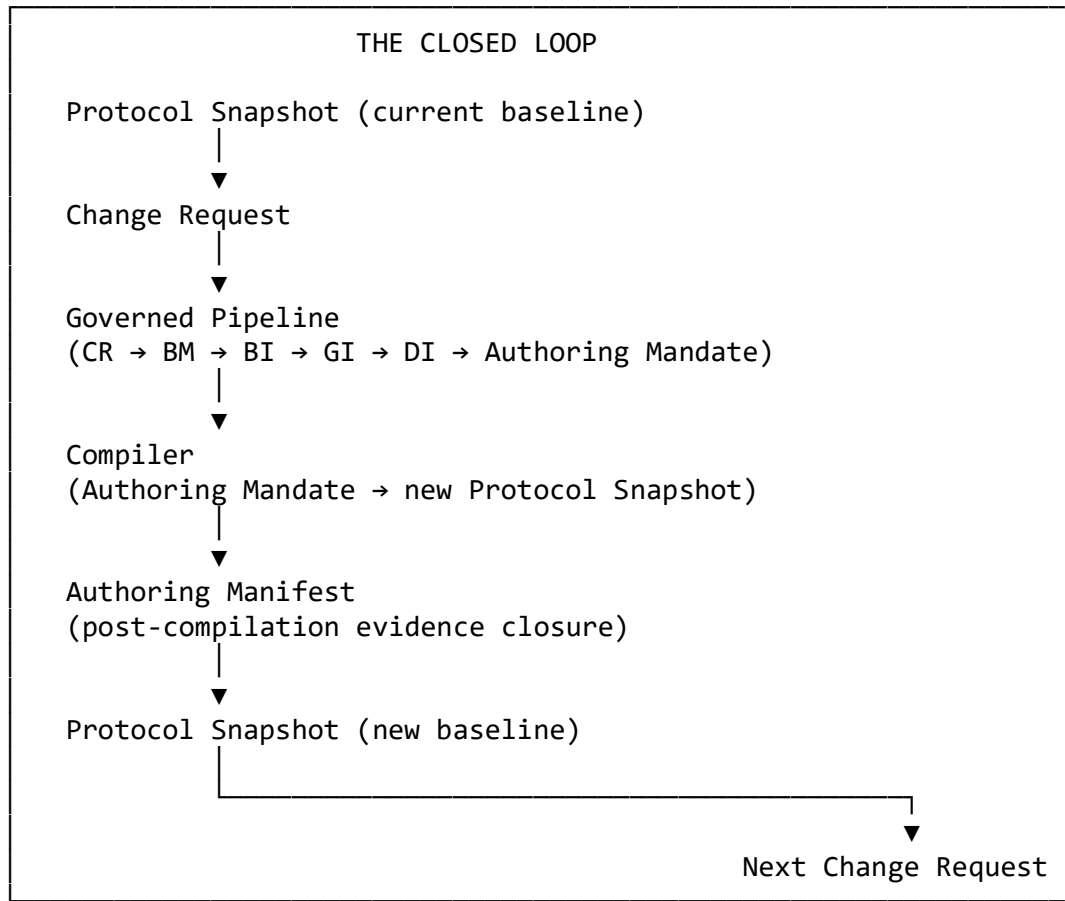
Stated as a forward-looking objective rather than a settled law: **as protocol coverage increases, the implementation change required to introduce new capability should decrease.** Each governed artifact already in the snapshot is leverage the next change reuses rather than rebuilds; at sufficient coverage, new capability becomes predominantly declarative composition over existing governed parts.

This objective is reinforced from a second, independent direction by the change-management authoring experiment — not only by runtime composition. There, the unit of authoring lifted from machine syntax to a validated structured contract that a deterministic renderer expands, repeating the same abstraction-lift PGS made from code to declaration and from snapshot to inspection. The convergence of two independent lines of evidence on the same objective is why it is **preserved here as an architectural objective of the PGS ecosystem** — and why it is deliberately **not yet constitutionalized**: it remains a Candidate Invariant pending cross-domain replication.

Seven governed change cycles. The concepts have not changed. The evidence has accumulated.

10. Closed-Loop Evolution

The four concepts established in sections 4 through 9 — nothing is greenfield, the governed pipeline, stage-enforced SoC, and governance as discovery — combine to define **closed-loop governed evolution**.



The loop has four structural properties that distinguish it from traditional change management:

Authority-bearing: Each stage document is a governance artifact. It is part of the dossier. It can be cited, reviewed, and compared against the Protocol Snapshot that it produced. Governance is not an informal conversation around the engineering process — it is the process.

Scope-contained: The Authoring Mandate identifies exactly which artifacts must be authored and in what order. The compiler validates that those artifacts are admissible. The scope of a CR cannot silently expand: expansion requires either a new CR or an explicit dossier revision that re-traverses the affected stages.

Evidence-complete: The Authoring Manifest closes the evidence chain. It records what was mandated, what was produced, what the conformance test results were, and what the

final snapshot state is. The dossier plus the Manifest is a complete, auditable record of a governed change.

Self-referential: The governed pipeline is itself subject to governance. Changes to the pipeline — new stage templates, new purity rules, new gate conditions — go through the same governed process. The pipeline is not infrastructure external to PGS. It is a governed concern within PGS.

This last property is what makes the loop genuinely closed. In traditional change management, the change management process is outside the system being managed. In PGS, the evolution process is governed by the same constitutional machinery that governs the system it produces.

**Traditional SDLC is open-loop: the output does not govern the next input.
Closed-loop governed evolution makes each output the baseline for the next governed change.**

11. Authority Invariance and Agent Engagement

Authority invariance with respect to the authoring actor. The pipeline’s most distinctive property is empirical, and it was demonstrated directly. One change cycle was executed by two very different authoring actors — a small local model (Qwen 3.5) and a frontier model (Claude Opus 4.8) — against an *identical* governance scaffold: the same stage templates, the same projection contracts, the same structural oracle, the same compiler rules. The drafts differed; their quality differed; their convergence differed. The governance did not — the oracle enforced identical rules, the projection contracts carried identical bounded context, and the human gates remained identical boundaries. What changed was the draft; what held was the governance. Most AI-assisted development systems embed authority *in the model* — the model’s judgment is the control. PGS externalizes it.

PGS is authority-invariant with respect to the authoring actor. A stage may be authored by a human, a local model, a frontier model, or a fully automated agent; the governing authority remains unchanged because authority resides in governed artifacts, projection contracts, structural validation, approval gates, and compiler enforcement rather than in the actor producing the draft. Automation may replace authoring labor, but it does not acquire decision-making authority.

Consequently, stronger models produce better drafts and weaker models produce poorer drafts, but neither alters the governance of the system. **The actor proposes; governance disposes.**

The claim is mechanically testable, not philosophical: the authoring actor possesses **no governing authority** — it cannot unilaterally alter governed state and cannot admit a change into the system. Only the compiler admits artifacts; only the structural oracle clears a stage; only a human closes a gate. A weaker actor cannot compromise governance — the oracle *rejects* its malformed identifiers, out-of-vocabulary values, and untraceable references rather than relaying them downstream — and a stronger actor merely reaches a clean draft sooner.

The PGS change-management loop is substrate-independent. Human authors, local models, frontier models, and future agent technologies participate through the same governed contracts and are evaluated by the same structural and compiler-based controls. Improvements in authoring capability increase productivity and draft quality but do not modify governance authority.

PGS is therefore not merely a protocol-driven SDLC; it is a **governance-first SDLC in which authoring is interchangeable but authority is not** — a property intended to outlast any particular authoring substrate.

The governed pipeline is designed to be agent-compatible. This is not the same as saying it requires AI. It does not. The pipeline works identically whether each stage is executed by a human governance engineer, an LLM-based agent, or any combination.

But the pipeline is **structurally well-suited** to agent engagement, for structural reasons that are direct consequences of the governed design:

Bounded context per stage: Each stage has a precisely defined scope of inquiry. The agent does not need to reason about the entire system — it reasons about the Business Model, or the Governance Intent, or the Design Intent. The scope boundary is structural, not navigated by convention.

Defined inputs: The canonical documentation set — Protocol Snapshot, PPS Snapshot, field manual, concept papers, and the dossier built so far — is the complete context. Nothing outside this set is required. An agent given this context has the same information that a human governance engineer has.

Stage-enforced stopping conditions: Discovery Saturation, GI Purity, BI Purity — these are verifiable properties. An agent can check them. It does not need to decide when enough analysis has been done; the saturation condition tells it.

Governance Decision Gates as handoff points: Gates are natural collaboration boundaries. The agent executes a stage; a human reviews the output at the Gate before the next stage begins. The protocol does not prescribe who must be human and who may be automated — the Gate is the boundary, not the identity of the actor at each stage.

Output is a governance artifact: The output of each stage is a structured document. It is version-controlled, referenced in the dossier, and compared against the Protocol Snapshot it eventually produces. The agent's output is subject to the same governance that a human's output is. There is no special agent path through the pipeline.

The agent-suitability claim has been stress-tested directly. One change cycle was deliberately executed by an automated agent with no prior exposure to PGS, given only the canonical documentation set through a declared context manifest. The agent produced a structurally complete dossier through every stage — and made architectural errors of a consistent and instructive kind: it designed against patterns it assumed rather than patterns the baseline declares, and it authored capabilities the baseline already contained. The subsequent quality-check pass, verifying every claim against the snapshot, corrected the dossier and — more importantly — converted each failure into an enforced stage rule: the elicitation contract, the verify-against-baseline obligation, and the search-before-authoring rule all hardened as a result. The experiment validated both halves of the model at once: the canonical set is sufficient context for a cold agent to do real governance work, and the pipeline's verification structure is what converts an agent's fluent-but-unverified output into governed knowledge. Agent capability and pipeline discipline are complements, not substitutes.

The design objective of the governed pipeline is **agent engagement** — the capacity to engage a capable agent (human or automated) as a participant whose output becomes a governed artifact. An agent that can execute a stage of the governed pipeline and produce

a durable governance artifact is doing governance, not generating documentation — but the authority lives in the artifact and its validation, never in the agent.

Whether that agent is powered by an LLM is an implementation detail. The pipeline will mature. Automation will improve. The governance model will remain unchanged.

The pipeline is LLM-agnostic by design and agent-suited by structure. These are not the same thing.

12. Future Direction

The governed pipeline currently operates as a governed human-agent process with external tooling. The pipeline stages are document artifacts in a dossier. The Authoring Mandate is the compiler's input. The Authoring Manifest closes the evidence record.

The natural future direction is for the pipeline itself to become a governed workflow:

`WF_PROCESS_CHANGE_REQUEST_V0`

When this exists, a Change Request submission is a protocol invocation. Each stage is a capability contract in the workflow topology. Governance Decision Gates are CC_ routing conditions. Discovery Saturation is a CT_ computation that returns a boolean outcome. The Authoring Mandate is a CS_ write to the compiler input surface.

At that point, the loop is fully closed not just conceptually but architecturally. The change management process is not a governed process that sits beside PGS — it is a governed workflow inside PGS. Every Change Request that alters the Protocol Snapshot is itself executed by the Protocol Snapshot it is about to change.

This is self-referential in a precise and productive sense: the governed system governs its own evolution using the same protocol machinery that governs everything else.

The completed Change Requests executed to date represent sufficient empirical validation of the conceptual model. They demonstrate that the pipeline produces consistent outputs, enforces stage purity, and accumulates governance knowledge. They also demonstrate that the model is substrate-independent: the pipeline was executed using a combination of human reasoning and LLM assistance, and the governing artifacts that resulted are identical in authority regardless of how they were produced.

The path from the current state to a fully governed `WF_PROCESS_CHANGE_REQUEST_V0` workflow is a governed change. It will be executed through the same pipeline described in this paper. The workflow described here is aspirational and not required for the validity of the conceptual model presented in this paper.

13. Conclusion

Protocol-Governed Systems established that behavior belongs in protocol, not in implementation. The compiler enforces what may exist. The runtime executes what was compiled. Protocol sovereignty is structural.

But a sovereign protocol that cannot evolve in a governed way is brittle sovereignty. Every change is an opportunity for protocol sovereignty to be compromised — by informal decisions, by implementation-laden requirements, by undocumented design rationale, by gradual drift between the declared system and the built system.

Closed-loop governed evolution closes this vulnerability. It applies the same principles to evolution that PGS applies to construction and execution:

- **Nothing is greenfield** — every change modifies a governed baseline; if the Protocol Snapshot does not change, the system is invariant
- **Stage-enforced SoC** — requirements cannot leak implementation decisions; the pipeline structure forbids it
- **Governance as discovery** — the pipeline produces architectural knowledge, not just documentation
- **The canonical documentation set** — the complete, implementation-free oracle for any change agent
- **The Governance Dividend** — governed evolution accumulates; each cycle makes the next cycle cheaper

These properties are direct consequences of SDLC inversion. Once the architecture is inverted — once behavior lives in protocol and not in implementation — the question is not whether to govern evolution, but how. The governed pipeline is the answer: a staged, authority-bearing, evidence-complete process that produces a compiler-ready artifact specification without ever introducing implementation decisions into the governance record.

The Protocol Snapshot is the governed artifact that construction produces. The governed pipeline is the governed artifact that evolution produces. Both are authoritative. Both are immutable once attested. Both are the product of a process that enforces the same sovereignty principles that make PGS a distinct architectural model.

PGS governs construction. PGS governs execution. Closed-loop governed evolution governs change. The loop is now closed.

Appendix A: Key Terms

Change Request (CR): The initiating artifact of a governed change cycle, produced by Stage 1. Declares the classification of the change together with the elicited problem, outcome, known facts, and explicit deferrals. Every dossier begins with a CR.

Dossier: The unit of governed change. A single evidence chain containing every stage document for one Change Request. Organized by domain and subdomain. Not a document type hierarchy — a governed artifact with temporal structure.

Governed Pipeline: The staged progression from Change Request through Business Model, Business Intent, Governance Intent, and Design Intent to the Authoring Mandate. Each stage is a bounded scope of inquiry; scope boundaries are enforced continuously as purity rules, and two Governance Decision Gates position human approval where the design is whole.

Elicitation Contract: The structural feature that opens every stage: a set of crisp questions addressed to the human, each paired with the declared intent for its answer — how the answer will be used in the document about to be drafted. An unanswered question is an open gap, never license to assume.

Business Model (BM): The canonical artifact of a CR. The complete model of actors, entities, resources, events, and their relationships — scoped to the change being governed. All downstream stages are projections of the Business Model.

Business Intent (BI): A projection of the scoped Business Model that captures behavioral intent — what the system must do — without implementation decisions. Provisional capability names are permitted; binding identifiers, store paths, module references, and build order are not (BI Purity).

Governance Intent (GI): Declares WHERE the governed change lives: domain, subdomain, ownership boundaries, and federation implications. Subject to GI Purity: no new artifact codes, no store paths, no build order. Existing baseline artifacts may be cited as evidence.

Design Intent (DI): Declares HOW the governed change will be expressed: binding identifier assignment, artifact family mapping, execution topology, schemas, versioning. Follows GI closure. Does not declare build order. Gate 1 (Design Approval) closes here.

Authoring Mandate: The governance-approved compiler input, produced by Stage 7. A structured specification of exactly what must be authored — artifacts, conformance tests, validation criteria — in a build sequence topologically sorted by dependency. The mandate is a derivation from the Design Intent, not a new decision, and must reconcile with it exactly. Gate 2 (Mandate Approval) closes here, locking the dossier.

Authoring Manifest: The evidence record that closes the dossier. Created as an empty baseline at Gate 2 and populated during and after authoring: what was mandated, what

was produced, approved deviations, discoveries, conformance test results, final snapshot state, and the cycle's methodology lessons. Manifest approval is CR closure.

Governance Decision Gate: A human approval point at which the dossier is reviewed as a body and locked. There are two: Gate 1 (Design Approval, closing Stage 6b) and Gate 2 (Mandate Approval, closing Stage 7). Distinct from scope boundaries, which are semantic locks between WHAT, WHERE, HOW, and IN WHAT ORDER, enforced continuously as purity rules rather than by approval.

Discovery Saturation: The stopping condition for the Analysis Loop. Reached when simultaneously: no unresolved CRITICAL gaps remain, no open analyst questions remain, and the dependency graph did not expand in the last review pass.

BI Purity: The invariant that Business Intent contains only behavioral declarations — no implementation vocabulary. Violation: any artifact family code, store name, or build order appearing in a BI document.

GI Purity: The invariant that Governance Intent contains only WHERE declarations — no new artifact codes, no store paths, no build order. The most commonly violated purity rule and the most important to enforce.

Purity Ladder: The vocabulary discipline that structures the entire pipeline: each stage admits exactly one more vocabulary class than the stage before it. Business language only through discovery (Stages 1–4); provisional capability names at Business Intent (Stage 5); placement without naming at Governance Intent (Stage 6); binding identifiers at Design Intent (Stage 6b); build order at the Authoring Mandate (Stage 7). One exception holds at every rung: artifacts that already exist in the governed baseline may be cited by their exact identifiers as evidence — citing the baseline is observation, not design.

Canonical Documentation Set: The complete, implementation-free oracle for governed change: Protocol Snapshot, PPS Snapshot, field manual, and concept papers. No implementation source code is required or included.

Governance Dividend: The accumulated property of the governed pipeline over multiple CRs: reduced change surface, increased predictability, and architectural leverage from formalized governance rules.

Governance as Discovery: The property that the governed pipeline produces architectural knowledge — boundaries, rules, and structural decisions — that was not specified in the Change Request and would not have emerged without the governed process.

Closed-Loop Evolution: The architectural property that the output of each change cycle — the new Protocol Snapshot and the completed dossier — becomes the authoritative baseline for the next change cycle. Evolution is not external to the governed system; it is the mechanism by which the governed system changes itself.

Agent Engagement: The design property that the governed pipeline provides a structured, bounded, evidence-bearing interface for any change agent — human, automated, or hybrid — to execute each stage with the same governance authority.

Appendix B: Reference Implementation Notes

The conceptual model presented in this paper has been realized in the open-source Protocol-Governed Systems (PGS) reference implementation available on GitHub:

PGS Workspace Repository

The governed change pipeline is implemented as `FB_CHANGE_MGMT` — a first-class governance boundary within `pgs_governance`, with its own constitution, dossier artifact templates, and lifecycle declarations. The `pgs_change_mgmt` repository contains the pipeline implementation: stage templates, dossier directory structure, and the pipeline execution tooling.

Seven Change Requests have entered the pipeline as of the time of publication — six executed through authoring into the compiled snapshot, one at design approval:

1. **blockchain::consensus_pos** — the first and most extensively documented cycle; produced 16 mandated authoring actions, 77/77 conformance PASS, and VALID snapshot status
2. **blockchain::block** — peer subdomain declared during `consensus_pos` GI stage; governed independently as a CR
3. **blockchain::data_model** — blockchain data model governed change; authoring mandate complete
4. **blockchain::consensus_propose** — governed block proposal: proposer selection, block formation, and consensus round recording
5. **blockchain::mempool** — governed staging of pending transactions; authoring manifest approved with full end-to-end regression
6. **blockchain::orchestration** — governed simulation and consensus-loop coordination; the source of the workflow-invocation and dedicated-storage-structure precedents
7. **blockchain::chain** — canonical chain and genesis bootstrap; executed as a deliberate agent stress test of the pipeline (Section 11) and at design approval as of this writing

The examples, governance rules, and architectural properties in this paper reflect the state of the project at the time of publication. The pipeline has been validated empirically. The conceptual model has remained materially stable across all completed cycles. The governance rules that emerged from the first cycle — GI Purity, BI Purity, Discovery Saturation, Authoring Manifest — have held across all subsequent cycles; the stage templates that operationalize them continue to absorb each cycle’s methodology lessons.

Since PGS is under active development, subsequent releases may extend the pipeline with additional stages, automate stage execution, or implement

`WF_PROCESS_CHANGE_REQUEST_V0` as a governed workflow. The conceptual properties documented in this paper — closed-loop evolution, nothing-is-greenfield, stage-enforced

SoC, and the Governance Dividend — are architectural properties of the PGS governance model, not features specific to any release.

For the latest documentation, releases, and implementation details, consult the project repository.

Appendix C: References

- Ganti, B. (2026). *Protocol-Governed Systems: Conceptual Model*. DOI: <https://doi.org/10.5281/zenodo.20300611>
- Ganti, B. (2026). *Protocol-Governed Systems: Compiler Conceptual Model*. DOI: <https://doi.org/10.5281/zenodo.20471804>
- Ganti, B. (2026). *Protocol-Governed Systems: Runtime Conceptual Model*. DOI: <https://doi.org/10.5281/zenodo.20478471>
- Ganti, B. (2026). *Protocol-Governed Systems: Architecture Inversion Concepts*. DOI: <https://doi.org/10.5281/zenodo.20497732>
- Beck, K., et al. (2001). *Manifesto for Agile Software Development*. agilemanifesto.org.
- Fowler, M. (2018). *Refactoring: Improving the Design of Existing Code* (2nd ed.). Addison-Wesley.
- ISO/IEC 20000-1:2018. *Information technology — Service management — Part 1: Service management system requirements*.
- AXELOS. (2019). *ITIL Foundation: ITIL 4 Edition*. TSO.
- Lamport, L. (1994). The Temporal Logic of Actions. *ACM Transactions on Programming Languages and Systems*, 16(3), 872–923.